

Cours 02 – Notion 3 : Encapsulation et Visibilité

2.3.1 Définitions de l'encapsulation et mise en contexte

L'encapsulation est une technique de design qui vise à assurer la cohérence des données en restreignant l'accès aux propriétés d'une classe. Des problèmes surviennent lorsque les attributs d'une classe sont visibles et modifiables à n'importe quel endroit dans une application.

```
public class Application
{
    public static void main(String[] args)
    {
        //Exemple d'appel pour un constructeur par initialisation
        Point2D pointA = new Point2D(0, 5);
        Point2D pointB = new Point2D(3, 1);

        Force force2 = new Force(pointA, pointB, "Fab");
        force2.module = 1500;
    }
}
```

force2 = {Force@724}

- > f nom = "Fab"
- f positionX = 0.0
- f positionY = 5.0
- f module = 0.0
- f orientation = -0.9272952180016122
- f estConnue = false
- f coefficient = 0.0

force2 = {Force@724}

- > f nom = "Fab"
- f positionX = 0.0
- f positionY = 5.0
- f module = 1500.0
- f orientation = -0.9272952180016122
- f estConnue = false
- f coefficient = 0.0

Dans la description initiale que nous avons faite de cette classe, nous avons affirmé que le champ "estConnue" devait rester à `false` tant que le module d'une classe n'avait pas été déterminé. Alors, si une classe ou un programmeur fixe le module d'une instance, il a l'obligation, par design, de modifier le champ "estConnue". Dans l'état actuel de la classe, aucun mécanisme n'assure que ce sera fait.

2.3.2 Implication de l'encapsulation

Lorsque l'encapsulation est utilisée comme technique de design, le concepteur d'une classe doit :

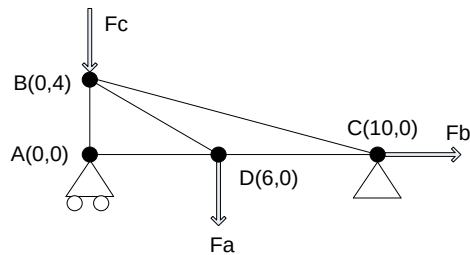
- Déclarer tous les attributs d'une classe comme étant privés.
- Donner accès aux attributs privés par un ensemble de méthodes publiques.
 - Les méthodes mutatrices permettent de fixer la valeur des propriétés privées.
 - Les méthodes observatrices permettent de consulter la valeur des propriétés privées.

Ces deux groupes de méthodes seront expliqués dans la prochaine capsule.

Dans ce cours, vous devrez employer l'encapsulation pour chacune de vos classes.

2.3.3 Concept de visibilité

Prenons, l'exemple d'un treillis. Celui-ci est composée de plusieurs nœuds auxquels sont rattachés les membrures du treillis.



Nous pouvons modéliser les nœuds d'un treillis de la manière suivante :

```
public class Noeud
{
    public String nom;           //Le nom du noeud
    public int    positionX;     //La coordonnée en x où est situé le nœud.
    public int    positionY;     //La coordonnée en y où est situé le nœud.
    public int    typeAppui;     //Détermine si le nœud est fixé à un appui et
                                //la nature de cet appui.
}
```

Ainsi le nœud A est un appui de type "rouleau", alors que le nœud B est un appui de type "goupille". Les nœuds B et D sont des nœuds réguliers qui ne sont pas des appuis. Ses quatre nœuds peuvent être construit à l'aide d'un constructeur spécialisé.

```
Nœud n1 = new Noeud('A', new Point2D( 0,0), 0);           // 0 --> Rouleau
Nœud n2 = new Noeud('B', new Point2D( 0,4), -1);          //-1 --> Sans Appui
Nœud n3 = new Noeud('C', new Point2D(10,0), 1);           // 1 --> Goupille
Nœud n4 = new Noeud('D', new Point2D( 6,0), -1);          //-1 --> Sans Appui
```

Les propriétés et les méthodes publiques d'une classe sont visibles et accessibles à n'importe quel moment dans une application. Ainsi, il est possible dans la classe Treillis, de consulter le type de nœud traité afin de savoir si des forces de réactions aux appuis doivent être générées.

```
104 -         if(this.noeuds[i].typeAppui == GOUPILLE || this.noeuds[i].typeAppui == ENCASTREMENT)
105             //On crée une nouvelle force afin de représenter la force trouvée.
106             //On ajoute la force à celle retournée.
107             disp(this.noeuds[i]);
108         end
109
```

```
▼ n = {Noeud@725}
> f nom = "C"
f positionX = 10 (0xA)
f positionY = 0 (0x0)
f typeAppui = 1 (0x1)
```

On remarque que dans cette situation (non désirée), le "Treillis" a accès aux données des nœuds et peut les utiliser comme bon lui semble.

Les propriétés et les méthodes privée d'une classe sont visibles et accessibles seulement à l'intérieur de la classe où elles ont été déclarées.

Ainsi, en utilisant une technique d'encapsulation adéquate, les propriétés de la classe nœud doivent être privées.

```
public class Noeud
{
    public private String nom; //Le nom du noeud
    public private int positionX; //La coordonnée en x où est situé le nœud.
    public private int positionY; //La coordonnée en y où est situé le nœud.
    public private int typeAppui; //Détermine si le nœud est fixé à un appui et
    //la nature de cet appui.
}
```

C:\Users\Francis\Desktop\INF111\PremierProgrammage\src\Application.java:13:13
java: typeAppui has private access in Noeud

Ce qui oblige la classe "Treillis" à utiliser une méthode publique pour consulter la propriété "typeAppui".

```
//On obtient le statut du noeud.
typeNoeud = this.noeuds[i].getTypeAppui();
if (typeNoeud == GOUPILLE || typeNoeud == ENCASTREMENT )
```

2.3.4 Autres exemples

Reprenons l'exemple illustré dans la mise en contexte de cette capsule, la modification du module, qui est une propriété privée de la force, doit se faire par l'utilisation d'une méthode privée.

```
public class Application
{
    public static void main(String[] args)
    {
        //Exemple d'appel pour un constructeur par initialisation
        Point2D pointA = new Point2D(0, 5);
        Point2D pointB = new Point2D(3, 1);

        Force force2 = new Force(pointA, pointB, "Fab");
        force2.setModule(1500);
    }
}
```

force2 = {Force@724} > f nom = "Fab" f positionX = 0.0 f positionY = 5.0 f module = 0.0 f orientation = -0.9272952180016122 f estConnue = false f coefficient = 0.0	force2 = {Force@723} > f nom = "Fab" f positionX = 0.0 f positionY = 5.0 f module = 1500.0 f orientation = -0.9272952180016122 f estConnue = true f coefficient = 0.0
--	--

Voici le code que devrait avoir la méthode "setModule".

```
// On modifie le module de la force en gardant les propriétés cohérentes
public void setModule(double module)
{
    this.module = module;
    this.estConnue = true;
}
```